LAPORAN PRAKTIKUM DASAR PEMROGRAMAN

PERTEMUAN – 8 : Perulangan 2



Nama : Muhammad Alif Sya’bani

NIM : 264107020163

Prodi : D IV Teknik Informatika

Kelas : 1B

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026/2027

Jobsheet 8: Perulangan 2

1. Tujuan

• Mahasiswa memahami konsep perulangan bersarang pada algoritma.

• Mahasiswa memahami alur perulangan bersarang.

• Mahasiswa mengenal sintaks perulangan bersarang.

• Mahasiswa mampu menerapkan perulangan bersarang untuk menyelesaikan studi kasus sederhana.

2. Praktikum

2.1 Percobaan 1: Review Perulangan 1

Program menampilkan karakter * sebanyak n kali secara mendatar menggunakan perulangan FOR. Input n diasumsikan berupa bilangan bulat positif.

Langkah Pengerjaan

Program disimpan dengan nama Star18.java. Scanner membaca nilai n, kemudian perulangan dimulai dari i = 1 sampai i <= n. Perintah print digunakan agar karakter bintang tetap berada pada satu baris.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca input dari keyboard.
n	int	Menentukan jumlah karakter bintang yang dicetak.
i	int	Penghitung perulangan dari 1 sampai n.

Implementasi ke Kode Program (Star18.java)

```
import java.util.Scanner;

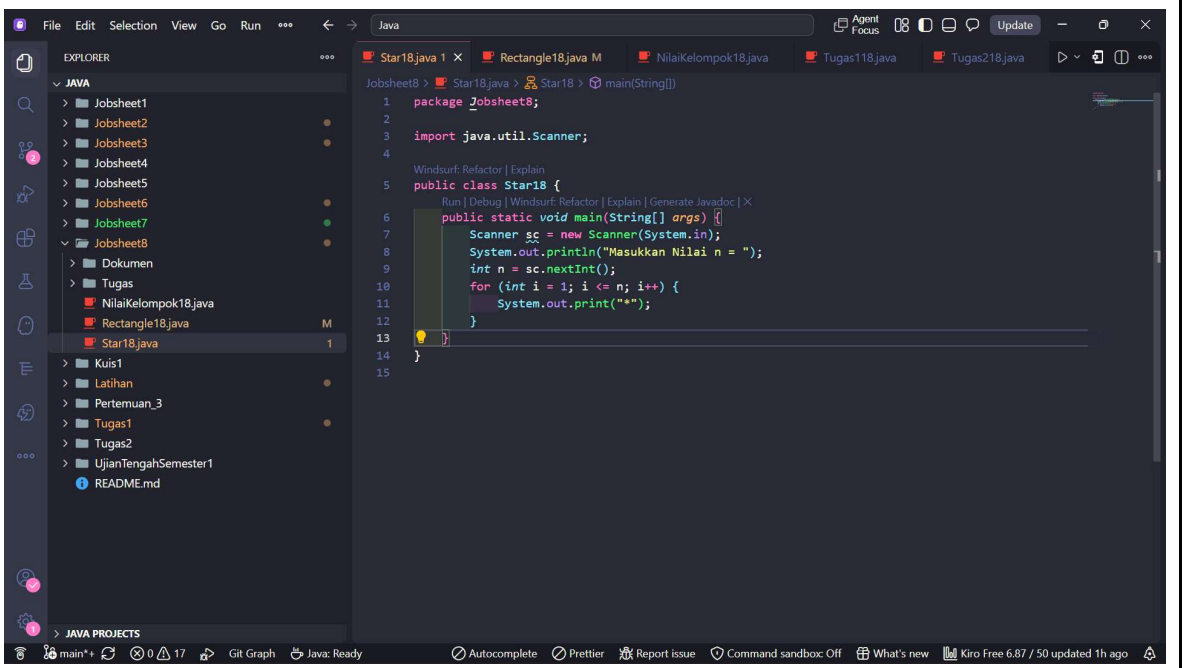
public class Star18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("*");
        }
        System.out.println();
        sc.close();
    }
}
```

Output Program (contoh pengujian)

```
Masukkan nilai n = 5
*****
```

Screenshot Hasil Program



Pertanyaan dan Jawaban

1. Jika inisialisasi i = 1 diubah menjadi i = 0 tanpa mengubah kondisi, berapa karakter * tercetak saat n = 5? Jelaskan.

Jawaban: Perulangan berjalan untuk i = 0, 1, 2, 3, 4, dan 5 sehingga mencetak 6 karakter *. Hal ini terjadi karena kondisi i <= n masih mencakup nilai 0 dan 5.

2. Jika kondisi i <= n diganti menjadi i < n, berapa kali perulangan dijalankan saat n = 5?

Jawaban: Perulangan dijalankan 4 kali, yaitu saat i bernilai 1, 2, 3, dan 4. Saat i = 5, kondisi i < n salah sehingga bintang kelima tidak dicetak.

3. Prediksi output kode for (int i = 2; i <= n; i += 2) saat n = 9 dan modifikasi untuk bilangan ganjil positif.

Jawaban: Output kode awal adalah 2 4 6 8. Untuk bilangan ganjil positif, gunakan for (int i = 1; i <= n; i += 2) { System.out.print(i + " "); }.

4. Mengapa i++ yang diganti i-- dengan kondisi i <= n membuat perulangan tidak berhenti dalam waktu wajar untuk n positif?

Jawaban: Nilai i terus berkurang dari 1, sehingga tetap memenuhi kondisi i <= n. Karena nilai i tidak bergerak menuju kondisi berhenti, perulangan terus berjalan. Versi ini tidak perlu dijalankan.

2.2 Percobaan 2: Pola Bintang Persegi Panjang (Nested FOR)

Percobaan ini menerapkan dua perulangan FOR. Outer loop mengatur banyaknya baris, sedangkan inner loop mengatur banyaknya bintang pada setiap baris.

Langkah Pengerjaan

Program disimpan dengan nama Rectangle18.java. Program membaca jumlah baris dan kolom. Setiap iterasi outer loop menjalankan inner loop sebanyak kolom kali, kemudian println memindahkan keluaran ke baris berikutnya.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca input.
baris	int	Menentukan jumlah baris persegi panjang.
kolom	int	Menentukan jumlah kolom atau bintang per baris.
i	int	Penghitung outer loop/baris.
j	int	Penghitung inner loop/kolom.

Implementasi ke Kode Program (Rectangle18.java)

```
import java.util.Scanner;

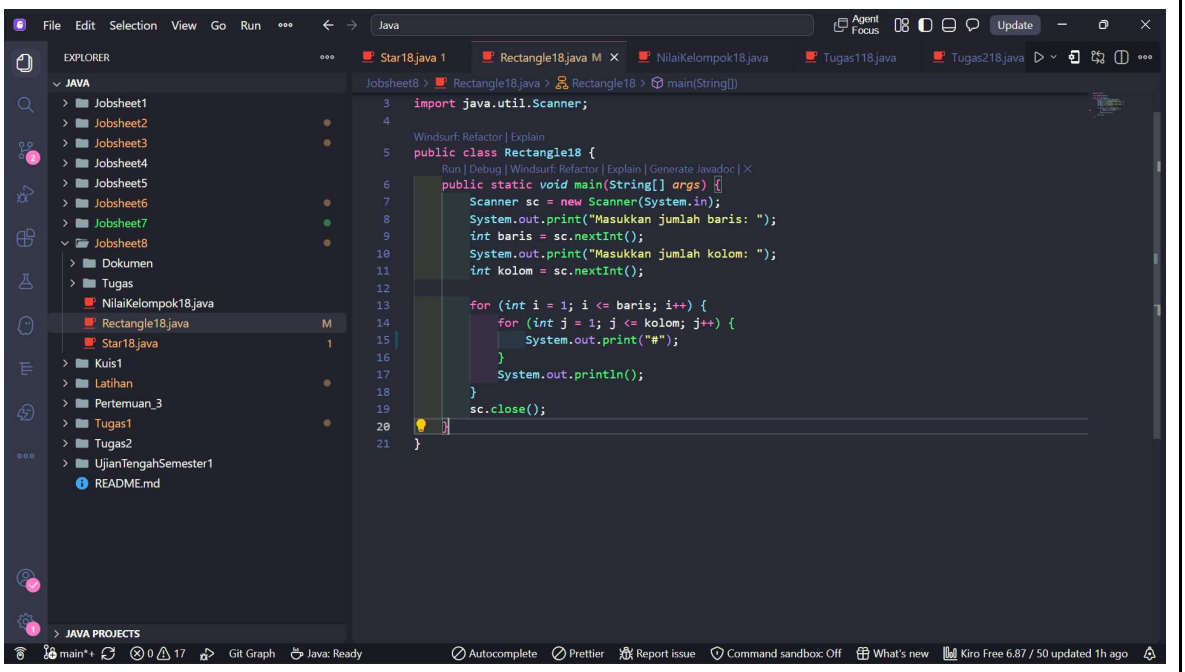
public class Rectangle18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumlah baris: ");
        int baris = sc.nextInt();
        System.out.print("Masukkan jumlah kolom: ");
        int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

Output Program (contoh pengujian)

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
******
******
```

Screenshot Hasil Program



Pertanyaan dan Jawaban

1. Tentukan fungsi variabel i dan j.

Jawaban: Variabel i adalah penghitung outer loop yang menentukan baris. Variabel j adalah penghitung inner loop yang menentukan jumlah karakter pada setiap baris.

2. Berapa kali outer loop, inner loop, dan print("*") dijalankan untuk baris = 4 dan kolom = 7?

Jawaban: Outer loop berjalan 4 kali. Inner loop berjalan 7 kali pada setiap baris atau 4 × 7 = 28 kali secara keseluruhan. Karena print("*") berada di inner loop, perintah tersebut juga dijalankan 28 kali.

3. Apa yang terjadi jika batas outer dan inner loop ditukar untuk baris = 3 dan kolom = 6?

Jawaban: Jika outer loop memakai kolom dan inner loop memakai baris, hasilnya menjadi 6 baris dengan masing-masing 3 bintang, yaitu persegi panjang yang orientasinya tertukar.

4. Jelaskan akibat penghapusan System.out.println() setelah inner loop.

Jawaban: Bintang dari semua iterasi outer loop akan dicetak berderet pada satu baris karena tidak ada perintah untuk pindah baris.

5. Modifikasi program agar karakter yang dicetak adalah #.

Jawaban: Ganti System.out.print("*"); menjadi System.out.print("#");. Struktur kedua perulangan tetap sama.

2.3 Percobaan 3: Debugging Bintang Segitiga – Nested WHILE

Kode awal sengaja belum menghasilkan pola segitiga dengan benar. Perbaikan dilakukan dengan tetap menggunakan nested WHILE, menambahkan pindah baris setelah inner loop, dan memastikan outer loop berjalan tepat n kali.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca nilai n.
n	int	Menentukan tinggi segitiga.
i	int	Penghitung baris pada outer loop.
j	int	Penghitung bintang pada inner loop.

Implementasi Kode Program yang Telah Diperbaiki

```
import java.util.Scanner;

public class Triangle18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        int i = 1;
        while (i <= n) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }

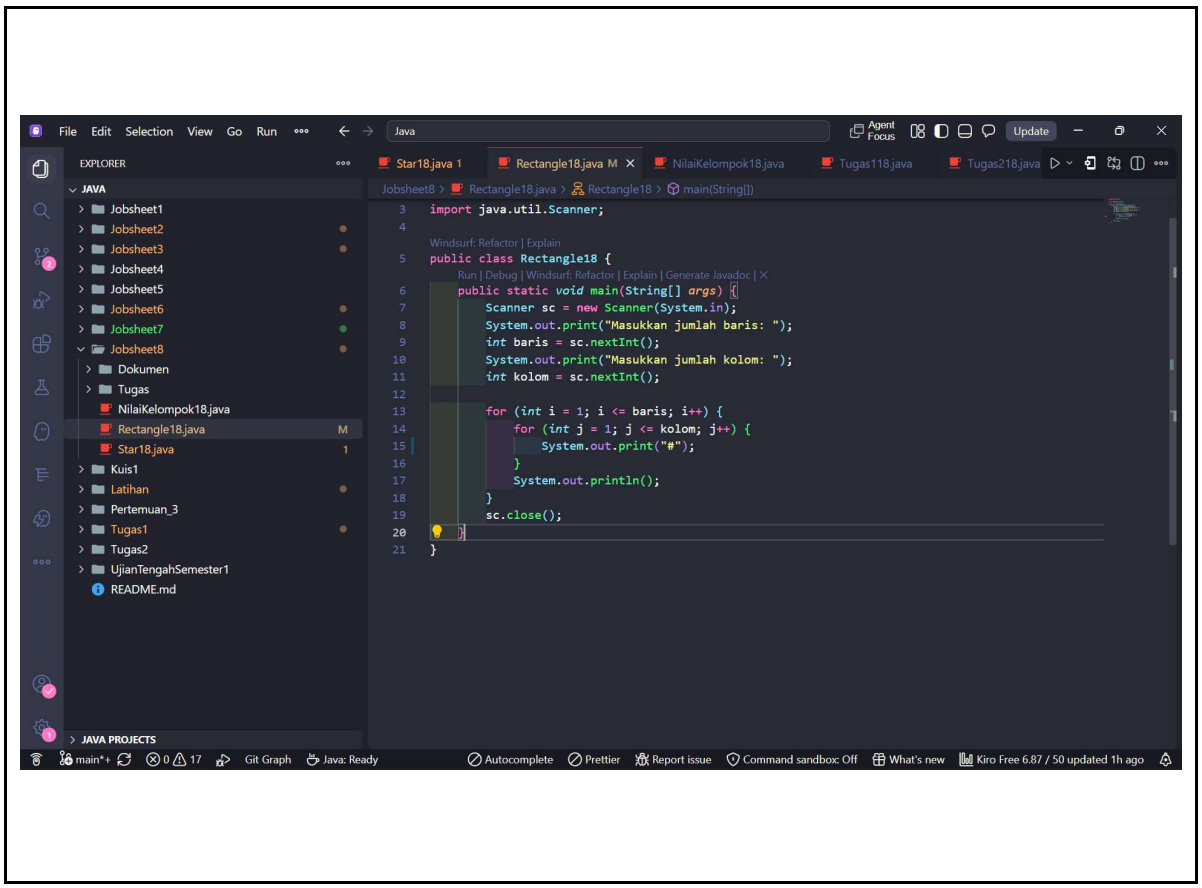
        i = n;
        while (i >= 1) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i--;
        }
        sc.close();
    }
}
```

Output Program (contoh pengujian n = 5)

```
Masukkan nilai n = 5
*
**
***
****
*****
```

```
*****
****
***
**
*
```

Screenshot Hasil Program



Pertanyaan dan Jawaban

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan.

Jawaban: Output awal mencetak seluruh bintang pada satu baris dan menjalankan outer loop mulai i = 0 sehingga terdapat iterasi tanpa bintang. Perbaikannya adalah menambahkan System.out.println() setelah inner loop dan mengubah inisialisasi/kondisi outer loop agar berjalan tepat n kali, misalnya i = 1 dan i <= n.

2. Jelaskan peran i dan j. Mengapa j harus diinisialisasi kembali ke 0 pada setiap iterasi outer loop?

Jawaban: i menentukan baris, sedangkan j menghitung bintang pada baris tersebut. j harus di-reset menjadi 0 agar setiap baris dimulai dari hitungan awal. Jika tidak di-reset, nilai j dapat tetap mencapai batas kondisi sehingga baris selanjutnya tidak mencetak bintang yang semestinya.

3. Jelaskan perubahan inisialisasi, kondisi, dan update outer loop untuk segitiga terbalik.

Jawaban: Outer loop dimulai dari i = n, memakai kondisi i >= 1, dan diperbarui dengan i--. Dengan demikian jumlah bintang berkurang satu pada setiap baris.

2.4 Percobaan 4: Studi Kasus Nilai Tugas Proyek Kelompok

Program menerima jumlah kelompok dan jumlah penilai untuk setiap kelompok. Nilai setiap penilai harus berada pada rentang 0–100. Program menghitung rata-rata tiap kelompok dan menentukan kelompok dengan rata-rata tertinggi. Perulangan yang digunakan adalah WHILE untuk kelompok, FOR untuk penilai, dan DO-WHILE untuk validasi nilai.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca input.
jumlahKelompok	int	Jumlah kelompok yang dinilai.
jumlahPenilai	int	Jumlah penilai untuk setiap kelompok.
nomorKelompok	int	Penghitung kelompok pada WHILE.
totalNilai	double	Akumulasi nilai penilai untuk kelompok saat ini.
nilai	int	Nilai dari satu penilai.
rataNilai	double	Rata-rata nilai kelompok saat ini.
rataTertinggi	double	Rata-rata tertinggi yang telah ditemukan.
kelompokTerbaik	int	Nomor kelompok dengan rata-rata tertinggi.
j	int	Penghitung penilai pada FOR.

Implementasi ke Kode Program (NilaiKelompok18.java)

```
import java.util.Scanner;

public class NilaiKelompok18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumlah kelompok : ");
        int jumlahKelompok = sc.nextInt();
        System.out.print("Masukkan jumlah penilai : ");
        int jumlahPenilai = sc.nextInt();

        if (jumlahKelompok <= 0 || jumlahPenilai <= 0) {
            System.out.println("Jumlah kelompok dan penilai harus > 0.");
        } else {
            int nomorKelompok = 1;
            double rataTertinggi = -1;
            int kelompokTerbaik = 0;

            while (nomorKelompok <= jumlahKelompok) {
                System.out.println("Kelompok " + nomorKelompok);
                double totalNilai = 0;

                for (int j = 1; j <= jumlahPenilai; j++) {
                    int nilai;
                    do {
                        System.out.print(" Nilai dari Penilai " + j + ": ");
                        nilai = sc.nextInt();
                        if (nilai < 0 || nilai > 100) {
                            System.out.println(" Nilai harus 0-100. Ulangi
```

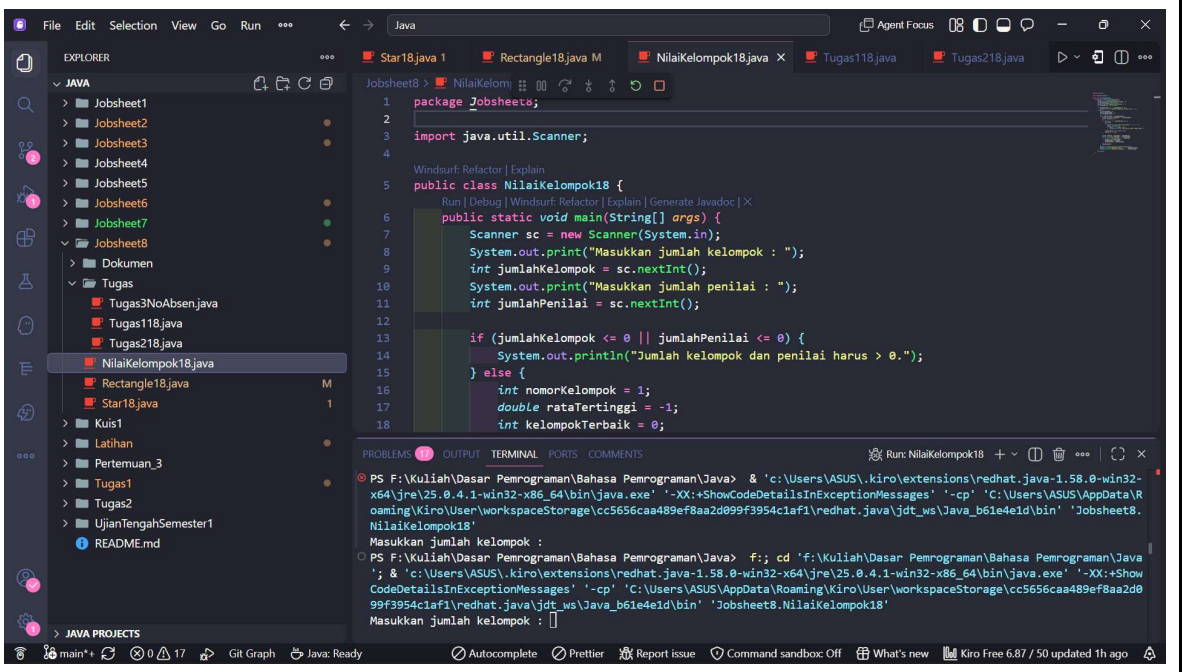
```
input!");
                        }
                    } while (nilai < 0 || nilai > 100);
                    totalNilai += nilai;
                }

                double rataNilai = totalNilai / jumlahPenilai;
                System.out.println("Kelompok " + nomorKelompok
                        + ": nilai rata-rata = " + rataNilai);
                if (rataNilai > rataTertinggi) {
                    rataTertinggi = rataNilai;
                    kelompokTerbaik = nomorKelompok;
                }
                nomorKelompok++;
            }
            System.out.println("==============================");
            System.out.println("Kelompok terbaik    : " + kelompokTerbaik);
            System.out.println("Rata-rata tertinggi : " + rataTertinggi);
        }
        sc.close();
    }
}
```

Output Program (contoh pengujian)

```
Masukkan jumlah kelompok : 3
Masukkan jumlah penilai : 2
Kelompok 1
 Nilai dari Penilai 1: 80
 Nilai dari Penilai 2: 90
Kelompok 1: nilai rata-rata = 85.0
Kelompok 2
 Nilai dari Penilai 1: 75
 Nilai dari Penilai 2: 105
 Nilai harus 0-100. Ulangi input!
 Nilai dari Penilai 2: 85
Kelompok 2: nilai rata-rata = 80.0
Kelompok 3
 Nilai dari Penilai 1: 90
 Nilai dari Penilai 2: 96
Kelompok 3: nilai rata-rata = 93.0
==============================
Kelompok terbaik    : 3
Rata-rata tertinggi : 93.0
```

Screenshot Hasil Program



Pertanyaan dan Jawaban

1. Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak boleh di-reset pada setiap kelompok?

Jawaban: totalNilai hanya menjumlahkan nilai milik satu kelompok, sehingga harus kembali ke 0 saat kelompok baru diproses. rataTertinggi digunakan untuk membandingkan semua kelompok sehingga nilainya harus dipertahankan sepanjang proses.

2. Untuk 3 kelompok dan 2 penilai tanpa input salah, berapa kali badan FOR dijalankan?

Jawaban: Sebanyak 3 × 2 = 6 kali, karena setiap kelompok menerima dua nilai penilai.

3. Apa yang terjadi jika totalNilai bertipe int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat?

Jawaban: Pembagian bilangan bulat dapat membuang bagian pecahan. Contohnya 161 / 2 menghasilkan 80, bukan 80,5. Menggunakan double untuk total atau mengonversi salah satu operand ke double menjaga hasil pecahan.

4. Mengapa rataTertinggi diberi nilai awal -1, bukan 0?

Jawaban: Karena nilai valid boleh 0. Jika rataTertinggi dimulai dari 0 dan semua kelompok memperoleh rata-rata 0, kondisi rataNilai > rataTertinggi tidak pernah benar sehingga kelompokTerbaik tetap 0, yang bukan nomor kelompok valid. Nilai -1 membuat kelompok pertama tetap tercatat.

5. Sebutkan tiga tingkat perulangan dan perannya.

Jawaban: WHILE terluar memproses setiap kelompok, FOR di tengah meminta nilai dari setiap penilai, dan DO-WHILE terdalam mengulang input selama nilai di luar rentang 0–100.

3. Tugas

3.1 Tugas 1: Pola Bilangan Bertingkat

Program mencetak pola bilangan dari 1 sampai nomor baris. Setelah pola selesai, program menampilkan total banyaknya angka yang telah dicetak.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca nilai n.
n	int	Jumlah baris pola.
i	int	Penghitung baris.
j	int	Angka yang dicetak pada setiap baris.
totalAngka	int	Menghitung seluruh angka yang dicetak.

Implementasi ke Kode Program (Tugas118.java)

```
import java.util.Scanner;

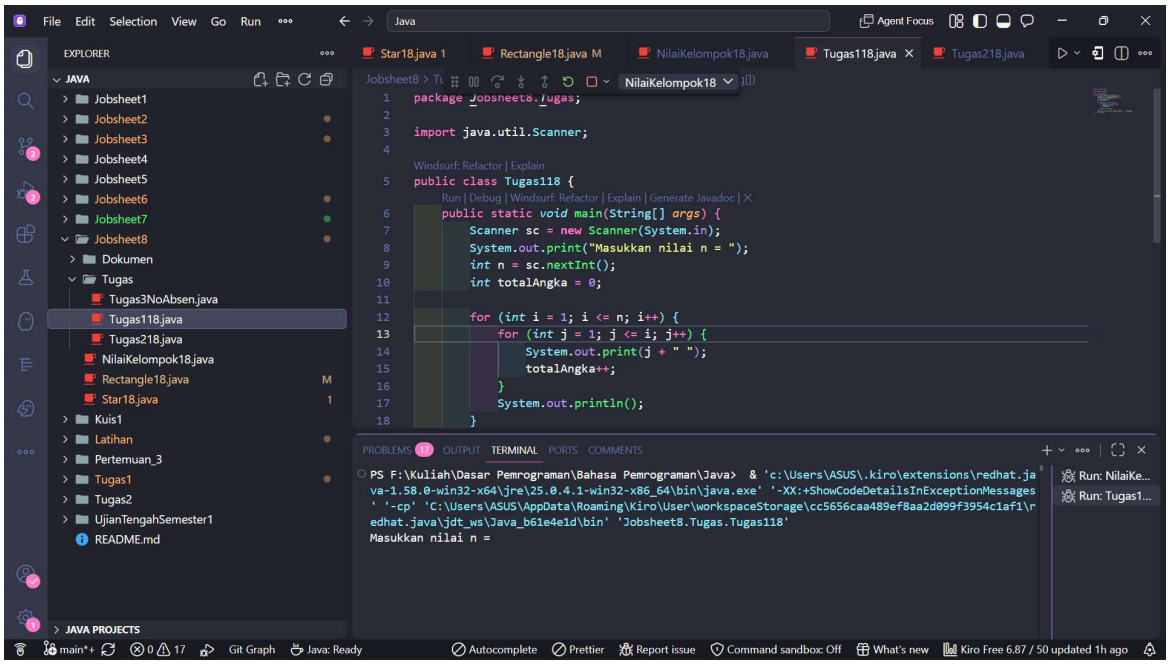
public class Tugas118 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();
        int totalAngka = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
                totalAngka++;
            }
            System.out.println();
        }
        System.out.println("Total angka tercetak: " + totalAngka);
        sc.close();
    }
}
```

Output Program (contoh pengujian n = 5)

```
Masukkan nilai n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
```

Screenshot Hasil Program



Penjelasan: Outer loop menentukan nomor baris, sedangkan inner loop mencetak bilangan mulai 1 sampai i. Variabel totalAngka bertambah satu setiap kali angka dicetak.

3.2 Tugas 2: Tabel Perkalian

Program menampilkan tabel perkalian berukuran n × n. Setiap sel berisi hasil perkalian nomor baris dan nomor kolom. Setelah tabel selesai, program menjumlahkan seluruh nilai pada tabel.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca nilai n.
n	int	Ukuran baris dan kolom tabel.
i	int	Nomor baris.
j	int	Nomor kolom.
hasil	int	Hasil perkalian i × j.
jumlah	long	Akumulasi semua hasil perkalian.

Implementasi ke Kode Program (Tugas218.java)

```
import java.util.Scanner;

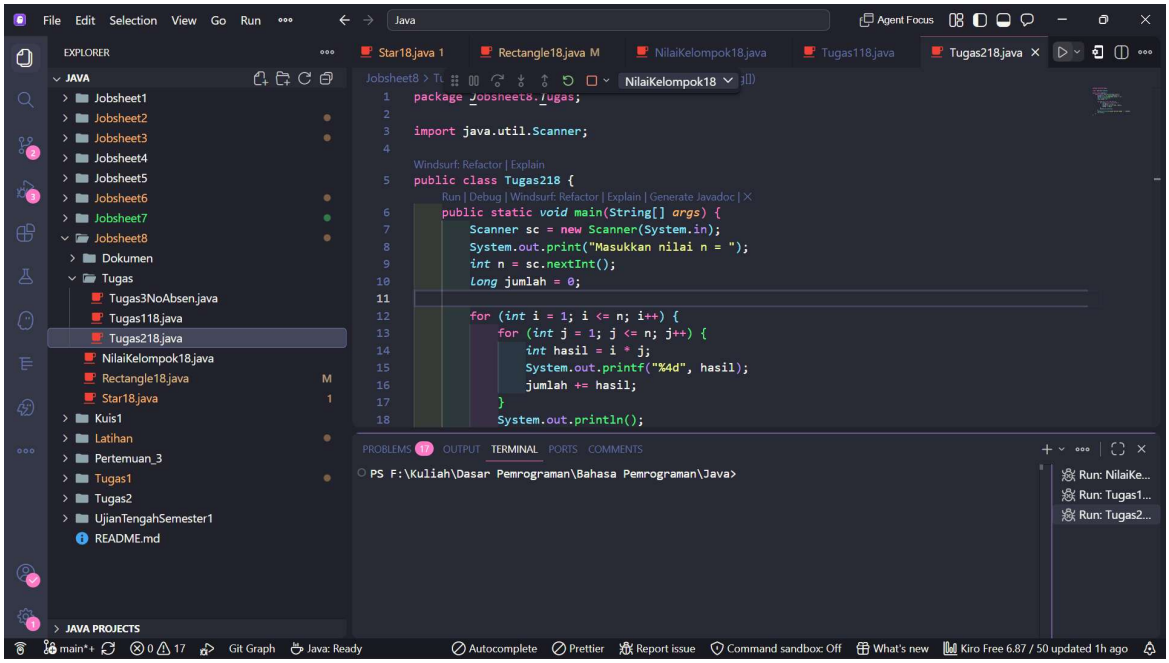
public class Tugas218 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();
        long jumlah = 0;
```

```
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                int hasil = i * j;
                System.out.printf("%4d", hasil);
                jumlah += hasil;
            }
            System.out.println();
        }
        System.out.println("Jumlah seluruh angka: " + jumlah);
        sc.close();
    }
}
```

Output Program (contoh pengujian n = 4)

```
Masukkan nilai n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
```

Screenshot Hasil Program



Penjelasan: Perkalian dilakukan untuk setiap pasangan baris dan kolom. Format %4d memberikan lebar empat karakter agar angka tersusun rata kanan. Variabel jumlah mengakumulasi seluruh hasil perkalian.

3.3 Tugas 3: Rekap Penjualan Beberapa Toko

Program meminta jumlah toko, jumlah pelanggan pada setiap toko, serta jumlah barang yang dibeli oleh setiap pelanggan. Program menghitung total pelanggan dan barang per toko, total barang seluruh toko, dan nomor toko dengan jumlah barang terbanyak. Jika terjadi seri, toko bernomor paling kecil dipertahankan.

Identifikasi Variabel

Variabel	Tipe Data	Kegunaan
sc	Scanner	Membaca input.
jumlahToko	int	Jumlah toko yang diproses.
toko	int	Penghitung nomor toko.
jumlahPelanggan	int	Jumlah pelanggan pada toko saat ini.
pelanggan	int	Penghitung pelanggan.
jumlahBarang	int	Barang yang dibeli seorang pelanggan.
totalBarangToko	int	Akumulasi barang satu toko.
totalPelanggan	int	Jumlah pelanggan yang diproses pada toko.
totalBarangSemua	long	Akumulasi barang seluruh toko.
barangTerbanyak	int	Jumlah barang terbesar pada satu toko.
tokoTerbanyak	int	Nomor toko dengan barang terbanyak.

Implementasi ke Kode Program (Tugas318.java)

```
import java.util.Scanner;

public class Tugas318 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int jumlahToko;
        do {
            System.out.print("Masukkan jumlah toko (minimal 1): ");
            jumlahToko = sc.nextInt();
            if (jumlahToko < 1) {
                System.out.println("Jumlah toko tidak boleh kurang dari 1.");
            }
        } while (jumlahToko < 1);

        long totalBarangSemua = 0;
        int barangTerbanyak = -1;
        int tokoTerbanyak = 0;

        for (int toko = 1; toko <= jumlahToko; toko++) {
            int jumlahPelanggan;
            do {
                System.out.print("Jumlah pelanggan Toko " + toko + ": ");
                jumlahPelanggan = sc.nextInt();
                if (jumlahPelanggan < 0) {
                    System.out.println("Jumlah pelanggan tidak boleh negatif.");
                }
            } while (jumlahPelanggan < 0);
```

```
            int totalBarangToko = 0;
            for (int pelanggan = 1; pelanggan <= jumlahPelanggan; pelanggan++) {
                int jumlahBarang;
                do {
                    System.out.print("Barang pelanggan " + pelanggan + " Toko "
                            + toko + ": ");
                    jumlahBarang = sc.nextInt();
                    if (jumlahBarang < 0) {
                        System.out.println("Jumlah barang tidak boleh
negatif.");
                    }
                } while (jumlahBarang < 0);
                totalBarangToko += jumlahBarang;
            }

            System.out.println("Toko " + toko + " - pelanggan: "
                    + jumlahPelanggan + ", total barang: " + totalBarangToko);
            totalBarangSemua += totalBarangToko;

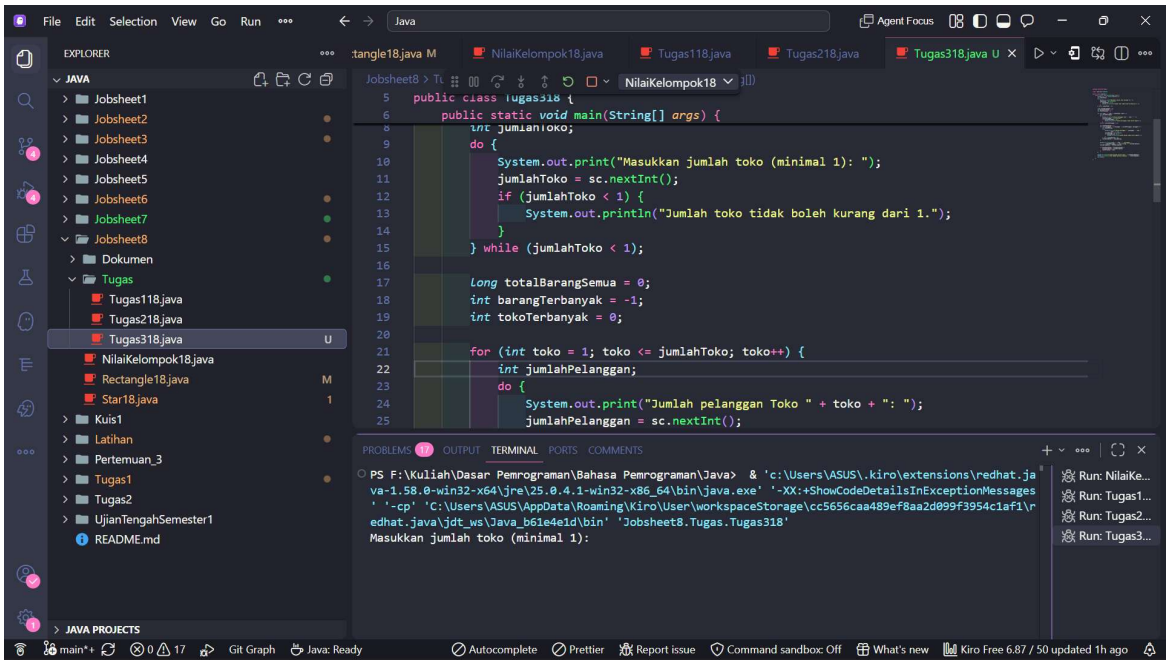
            if (totalBarangToko > barangTerbanyak) {
                barangTerbanyak = totalBarangToko;
                tokoTerbanyak = toko;
            }
        }

        System.out.println("Total barang seluruh toko: " + totalBarangSemua);
        System.out.println("Toko dengan barang terbanyak: " + tokoTerbanyak);
        sc.close();
    }
}
```

Output Program (contoh pengujian)

```
Masukkan jumlah toko (minimal 1): 2
Jumlah pelanggan Toko 1: 2
Barang pelanggan 1 Toko 1: 3
Barang pelanggan 2 Toko 1: 2
Toko 1 - pelanggan: 2, total barang: 5
Jumlah pelanggan Toko 2: 3
Barang pelanggan 1 Toko 2: 1
Barang pelanggan 2 Toko 2: 4
Barang pelanggan 3 Toko 2: 2
Toko 2 - pelanggan: 3, total barang: 7
Total barang seluruh toko: 12
Toko dengan barang terbanyak: 2
```

Screenshot Hasil Program



Penjelasan: Perulangan pertama memproses toko. Perulangan kedua memproses pelanggan di setiap toko. Jumlah barang setiap pelanggan diakumulasikan ke totalBarangToko, lalu nilai tersebut ditambahkan ke totalBarangSemua. Operator > membuat toko bernomor lebih kecil tetap terpilih jika jumlah barang tertinggi sama.

4. Kesimpulan

Berdasarkan Jobsheet 8 tentang perulangan bersarang, dapat disimpulkan bahwa nested loop memungkinkan satu perulangan dijalankan di dalam perulangan lain untuk menyelesaikan persoalan yang memiliki beberapa tingkat pengulangan.

• Pada pola bintang persegi panjang, outer loop mengatur baris dan inner loop mengatur kolom.

• Pada pola segitiga, variabel penghitung perlu diinisialisasi dan diperbarui dengan tepat serta baris perlu diakhiri dengan println agar pola terbentuk benar.

• Kombinasi WHILE, FOR, dan DO-WHILE dapat digunakan untuk memproses kelompok dan penilai sekaligus memvalidasi nilai.

• Nested loop juga dapat digunakan untuk membuat pola bilangan, tabel perkalian, serta rekap pelanggan dan barang pada beberapa toko.

• Variabel akumulasi perlu ditempatkan pada lingkup yang sesuai: total per kelompok atau toko di-reset untuk setiap entitas, sedangkan total keseluruhan dipertahankan sampai semua data diproses.

Catatan pengerjaan: kode dan keluaran yang dicantumkan merupakan contoh implementasi berdasarkan instruksi Jobsheet 8. Jalankan setiap program di Visual Studio Code, cocokkan hasil aktual, dan tambahkan tangkapan layar serta bukti commit/push jika diwajibkan sebelum laporan dikumpulkan.